

吴恩达 NLP 课程 3

目录

吴恩达 NLP 课程 3.....	1
1. 大纲介绍.....	3
2. 第一周（深度学习的基础知识）	4
2.1 前向传播的复习	4
2.2 Trax 模型	6
2.3 为什么推荐 Trax?	7
2.4 Trax 官方文档和源码（包含 JAX 库）	15
2.5 python 中的类介绍.....	16
2.6 Trax 中的图层	17
2.7 Trax 框架下的梯度下降	20
3. 第二周（N-gram 与序列模型）	22
3.1 传统语言模型	22
3.2 循环神经网络（RNN）Recurrent neural network	23
3.3 RNN 架构类型.....	24
3.4 简单 RNN 的教学	25
3.5 GRU（门控循环单元）	29
3.6 深层 RNN 和双向 RNN.....	31
4. 第三周.....	33
4.1 RNN 和梯度消失.....	33
4.2 LSTM	36

4.3 LSTM 体系结构.....	38
4.4 命名实体识别	41
5. 第四周.....	45
5.1 Siamese 网络简介	45
5.2 架构	46
5.3 代价函数	47
5.4 三胞胎?	48
5.5 代价函数的计算	50
5.6 分类与一次性学习	54
5.7 如何训练测试 Siamase 网络.....	54

1. 大纲介绍

- 第一周：神经网络基础知识
- 第二周：语言建模
- 第三周：命名实体识别
- 第四周：识别文本重复（相同的问题，不同的措辞）

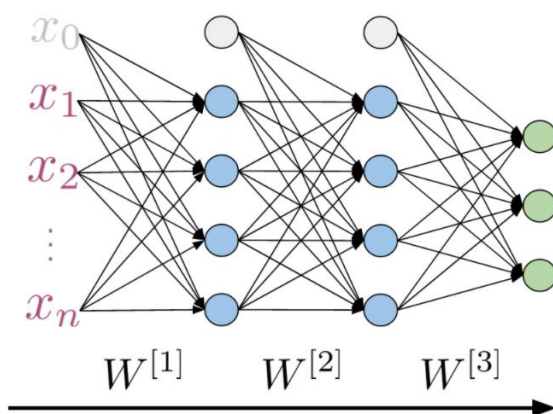
2. 第一周（深度网络的基础知识）

深度学习中的神经网络用于情感分析

2.1 前向传播的复习

Take some time to look at the equations:

Forward propagation



$a^{[i]}$ Activations ith layer

$$a^{[0]} = X$$

$$z^{[i]} = W^{[i]} a^{[i-1]}$$

$$a^{[i]} = g^{[i]}(z^{[i]})$$

与您以前的情感分析工作类似，您首先需要列出所有句子中的单词。接下来您将为其分配一个整数索引，然后为您的推文中的每个单词添加索引，使得您的每条推文都带有这样的向量。拥有所有向量之后，您将需要确定最大向量长度并用 0 填充向量以匹配最大长度的大小。此过程称为填充，可确保所有您的向量大小相同，即使您的推文没有那么长。

Initial Representation

Word	Number
a	1
able	2
about	3
...	...
hand	615
...	...
happy	621
...	...
zebra	1000

Tweet: This movie was almost good

[700 680 720 20 55]

↓ Padding

[700 680 720 20 55 0 0 0 0 0 0]

To match size of longest tweet

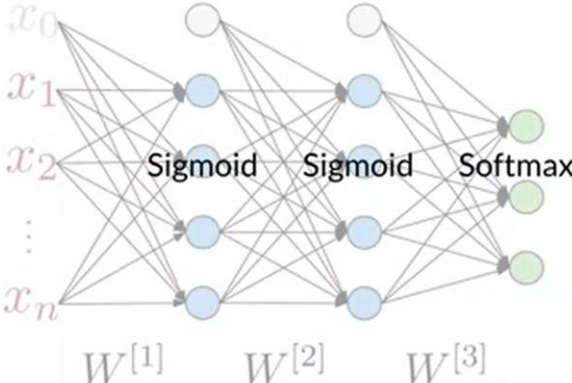
even if your tweets don't.

2.2 Trax 模型

现在的深度学习有很多开源框架，Trax 是最新的产品之一，它基于 TensorFlow。

优点：执行效率高（指定模型架构后可自动计算梯度等），可并行计算（支持 CPU，GPU 和 TPU）。

Neural Networks in Trax

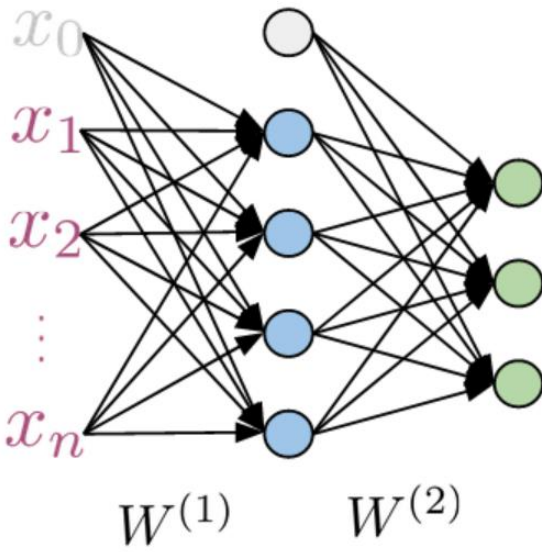


```
from trax import layers as tl
Model = tl.Serial(
    tl.Dense(4),
    tl.Sigmoid(),
    tl.Dense(4),
    tl.Sigmoid(),
    tl.Dense(3),
    tl.Softmax())
```

the computations are made
in your neural network.

试题检测：

Given the following structure, how many layers would your serial model have if implemented using trax?



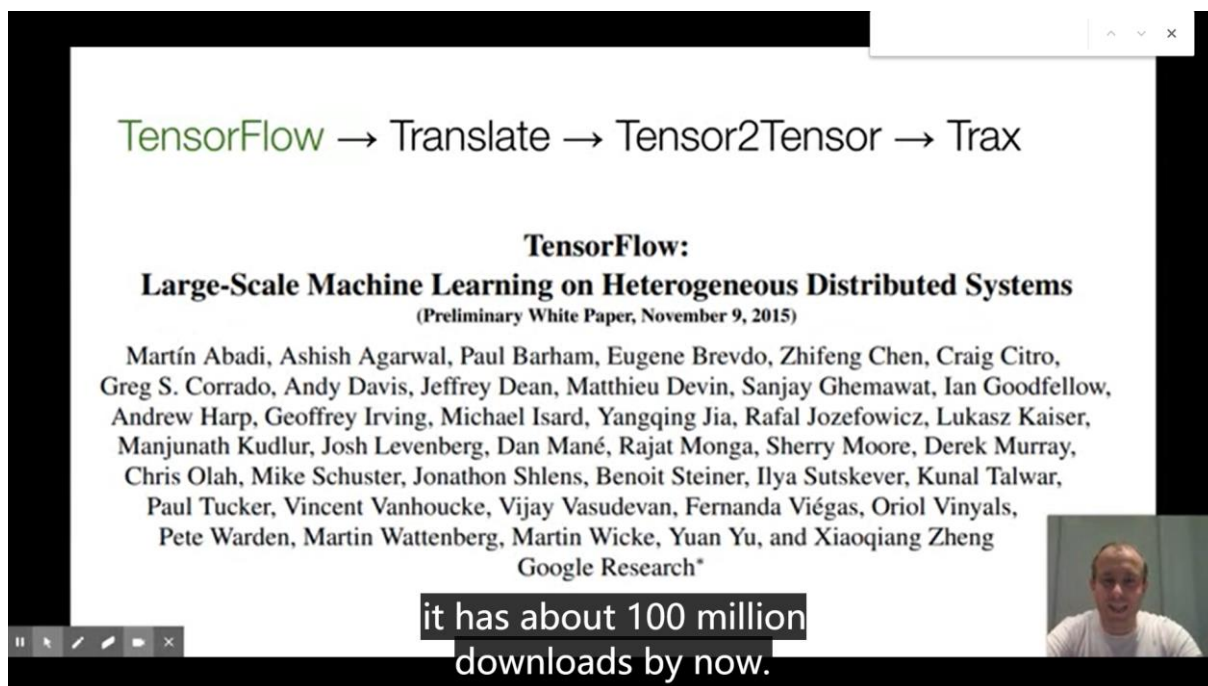
答案是 4

2.3 为什么推荐 Trax?

卢卡斯的个人介绍和推荐理由：

你好。我的名字是卢卡斯，我想在这段视频中告诉你我们为什么要制作机器学习库 Trax。这对我来说是一个私人故事。我在谷歌已经七年了。我是谷歌大脑小组的研究员。但在我做研究员之前，我是一名软件工程师。我参与过很多机器学习项目和框架。我的这次旅行在特拉克斯图书馆结束了。我相信 Trax 是目前学习和生产机器学习研究和机器学习模型的最佳图书馆，尤其是序列模型、像 transformer 这样的模型以及用于自然语言处理的模型。我相信这是因为我经历了一次个人旅行才来到这里的。我会告诉你一些关于我自己和我是如何来到这里的，然后我会告诉你为什么我认为 Trax 是目前机器学习中最棒的东西，尤其是在自然语言处理中。

我的机器学习和机器学习框架之旅始于 2014-15 年，当时我们正在制作 TensorFlow。你可能知道，TensorFlow 是一个大型的机器学习系统，到目前为止已经有 1 亿的下载量。2015 年 11 月发布。当我们发布它的时候，对我们所有人来说都是一个非常激动的时刻。那时，我们还不确定深度学习是否会像以前那样大。我们不确定会有多少用户。我们想做的是一个主要是非常快速的系统，可以运行分布式机器学习系统，大规模快速培训，重点是速度！



The image shows a presentation slide titled "TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems" with a list of authors from Google Research. Above the title is a flow diagram: TensorFlow → Translate → Tensor2Tensor → Trax. Below the authors' names, a video player interface is visible, showing a man speaking and a subtitle that reads "it has about 100 million downloads by now." The video player has standard controls like play, pause, and volume.

TensorFlow → Translate → Tensor2Tensor → Trax

TensorFlow:
Large-Scale Machine Learning on Heterogeneous Distributed Systems
(Preliminary White Paper, November 9, 2015)

Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dan Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng
Google Research*

it has about 100 million
downloads by now.

第二个重点是使编程不是阅读器的系统变得容易，但这不是最重要的事情。在发布 TensorFlow 之后，我研究了机器翻译，特别是谷歌的神经机器翻译系统。这是 Google 翻译团队使用的第一个使用深度序列模型的系统，它实际上是作为产品发布的。如今，谷歌正在处理所有的谷歌翻译。我们所有的语言都有一个神经模型。它从 LSTMs 和 RNN 模型开始，现在已经有很多变种。我们在 2016 年发布了基于 TensorFlow 框架的产品。这些模型，真是太棒了。它们比以前基于短语的翻译模型要好得多，但是它们需要很长时间来训练。当时他们在 gpu 集群上训练了好几天。这对于其他任何人来说都是不现实的，除了谷歌。这仅仅是因为我们有这个张力流系统，一大群能很好运用该集群的工程师，我们每天都在训练。太棒了。但我觉得这并不令人满意，因为没有人能做到这一点，尤其是在大学。

TensorFlow → Translate → **Tensor2Tensor** → Trax

Accelerating Deep Learning Research with the Tensor2Tensor Library

Monday, June 19, 2017

Posted by Lukasz Kaiser, Senior Research Scientist, Google Brain Team

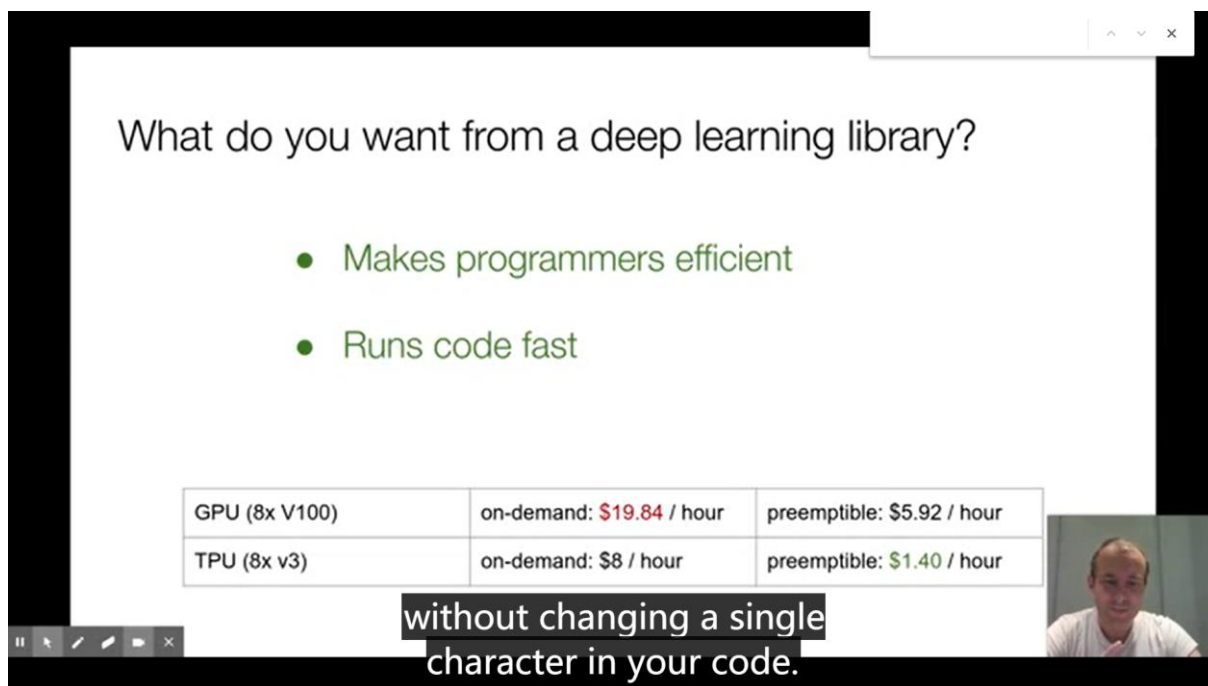
Translation Model	Training time	BLEU (difference from baseline)
Transformer (T2T)	3 days on 8 GPU	28.4 (+7.8)
SliceNet (T2T)	6 days on 32 GPUs	26.1 (+5.5)
GNMT + Mixture of Experts	1 day on 64 GPUs	26.0 (+5.4)
ConvS2S	18 days on 1 GPU	25.1 (+4.5)
GNMT	1 day on 96 GPUs	24.6 (+4.0)
ByteNet	8 days on 32 GPUs	23.8 (+3.2)
MOSES (phrase-based baseline)	N/A	20.6 (+0.0)

BLEU scores (https://www.tensorflow.org/tensor2tensor/)

we need to do even better,



我想改变这一点。为此，我们创建了 Tensor2Tensor 库。2017 年发布的 Tensor2Tensor 库的出发点是，我们应该让这种深度学习研究，特别是序列模型的研究，能够被广泛使用。这并不适用于这些大型 RNN 模型，但是在编写库时，我们创建了这个转换器模型。这台转换器让你的训练速度快得多，当时 DL 界受到了极大的冲击。那时的训练需要好几天，现在，在 8 个 GPU 系统上，几小时内不到一天就可以完成。您可以创建超越任何 RNN 模型的翻译模型。张量传感器库已经得到了广泛的应用，它被用于生产谷歌系统。世界上一些非常大的公司都在使用它，它已经导致了很多初创公司，他们知道这些基本上都是由于这个图书馆而存在的。你可以说，好吧，这已经完成了，这很好，但是，问题是，它变得越来越复杂，不好学习，而且很难做新的研究员。2018 年前后，我们决定是时候改进了。



What do you want from a deep learning library?

- Makes programmers efficient
- Runs code fast

GPU (8x V100)	on-demand: \$19.84 / hour	preemptible: \$5.92 / hour
TPU (8x v3)	on-demand: \$8 / hour	preemptible: \$1.40 / hour

without changing a single character in your code.

随着时间的推移，我们需要做得更好，这就是我们如何创建 Trax 的。Trax 是一个深度学习的库，专注于清晰的代码和速度。让我告诉你为什么，所以，如果你仔细考虑你想要从一个深度学习的图书馆里得到什么，有两件事是很重要的。你希望程序员效率高，代码运行速度快，这是因为花在你身上的是程序员的时间，以及运行培训代码所需的资金。程序员的时间很重要。你需要有效地使用它，但在深度学习中，你在培训大模型，而且这些也要花钱。例如，使用 8 个 gpu 点播，几乎每小时花费 20 美元。但使用可抢占的 8 个 TPU 可能只需 1.40 美元。在 Trax 中，可以使用一个或另一个，而不必更改代码中的单个字符。Trax 如何使程序员更有效率呢？嗯，它是自下而上重新设计的，以便于调试和理解。你可以直接阅读 Trax 代码并理解接下来会发生什么。在其他一些库中并不是这样，在 TensorFlow 中这种情况已经很不幸了。但是，你可以说，以前是这样，但是现在 TensorFlow，即使我们清理代码，它也需要向后兼容。它承载了这些年的发展，这是机器学习的疯狂错误。因为它是向后兼容的，所以它需要承载很多包袱。我们在 Trax 中所做的就是打破向后兼容性。这意味着你需要学习新的东西。这是有代价的。但是你这个代价中得到的是，这是一个全新设计的

库，它有四个模型，不仅仅是构建它们的原语，还有四个带有数据集绑定的模型，我们每天都对这些模型进行回归测试，因为我们使用这些库，所以我们知道这些怪物每天都在运行。它就像一种新的编程语言。

Trax makes programmers efficient

- Bottom-up clean re-design
- Easy to debug, you can read the code
- Full models with dataset bindings included
- Main models regression-tested daily

Is there a price to pay?

- Backwards compatibility

It's like a new programming language.

这是一个新事物，但它使你的生活更有效率。为了明确这一点，Adam 优化器是机器学习时间步长中最流行的优化器。在左边，你可以看到一张介绍数据的报纸的截图，大概有七行。下一步只是 Adam 实现和赞助的一部分，这实际上是最干净的一个，你需要知道更多的方法，你需要知道什么是参数组，你需要知道通过某种方式把参数键放入这些组中，你需要做七个键初始化和一些有条件的引入和其他的事情。在右侧，您可以看到 TensorFlow 和 Keras 中的 Adam 优化器，并且您将看到它甚至更长。你需要把它应用到资源变量和两个非研究变量上，你需要知道它们是什么。它们存在的原因是历史的。目前我们只使用资源变量，但我们必须支持那些使用旧的非研究变量的人。在 2020 年，你实际上不再需要很多东西，但它们必须在那里，用 TensorFlow 代码绘制。如果你看 Trax 代码，这是 Adam 和 Trax 的完整代码。这和报纸很相似。这就是重点。因为如果你在写一篇新论文，或者你正在学习，你想在框架

的代码中找到论文中的方程式，你可以在这里做。这就是 Trax 的好处。这个好处的代价是你正在使用一种新的东西。但是当你真正调试你的代码时，你会得到一个巨大的收获。在调试代码时，您将命中框架中的行。所以你实际上需要理解这些线，这意味着你需要理解所有这些 PyTorch 和所有这些张量流，如果你使用它们的话。但是在 Trax 中，你只需要理解这些 Trax 线。它更容易调试，这使程序员更有效率。现在，如果代码运行缓慢，这种效率就不值得那么多了。

The screenshot displays a video player with a presentation slide titled "Adam Optimizer". The slide contains the mathematical formulation of the Adam algorithm, followed by its implementation in Python. The Python code is a detailed, line-by-line translation of the mathematical steps, using comments to explain each operation. The video player interface includes standard controls like play, pause, and volume, and a small inset of the speaker in the bottom right corner.

Adam Optimizer

$m_0 \leftarrow 0$ (Initialize 1st moment vector)
 $v_0 \leftarrow 0$ (Initialize 2nd moment vector)
 $t \leftarrow 0$ (Initialize timestep)
while θ_t not converged **do**
 $t \leftarrow t + 1$
 $g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$ (Get gradients w.r.t. stochastic objective at timestep t)
 $m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ (Update biased first moment estimate)
 $v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ (Update biased second raw moment estimate)
 $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ (Compute bias-corrected first moment estimate)
 $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ (Compute bias-corrected second raw moment estimate)
 $\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ (Update parameters)
end while

The reason they exist is historical.

Trax Adam

```
class Adam(Optimizer):  
    """Adam optimizer."""  
  
    def init(self, param):  
        m = np.zeros_like(param)  
        v = np.zeros_like(param)  
        return m, v  
  
    def update(self, step, grads, param, slots, opt_params):  
        m, v = slots  
        learning_rate, b1, b2, eps = opt_params  
        m = (1 - b1) * grads + b1 * m # First moment estimate.  
        v = (1 - b2) * (grads ** 2) + b2 * v # Second moment estimate.  
        mhat = m / (1 - b1 ** (step + 1)) # Bias correction.  
        vhat = v / (1 - b2 ** (step + 1))  
        param = param - (  
            learning_rate * mhat / (np.sqrt(vhat) + eps)).astype(param.dtype)  
        return param, (m, v)
```

```

$$m_0 \leftarrow 0 \text{ (Initialize 1st moment vector)}$$

$$v_0 \leftarrow 0 \text{ (Initialize 2nd moment vector)}$$

$$t \leftarrow 0 \text{ (Initialize timestep)}$$
while  $\theta_t$  not converged do  
     $t \leftarrow t + 1$   
     $g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$  (Get gradients w.r.t. stochastic objective at timestep  $t$ )  
     $m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$  (Update biased first moment estimate)  
     $v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$  (Update biased second raw moment estimate)  
     $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$  (Compute bias-corrected first moment estimate)  
     $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$  (Compute bias-corrected second raw moment estimate)  
     $\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$  (Update parameters)  
end while
```

The price of this benefit



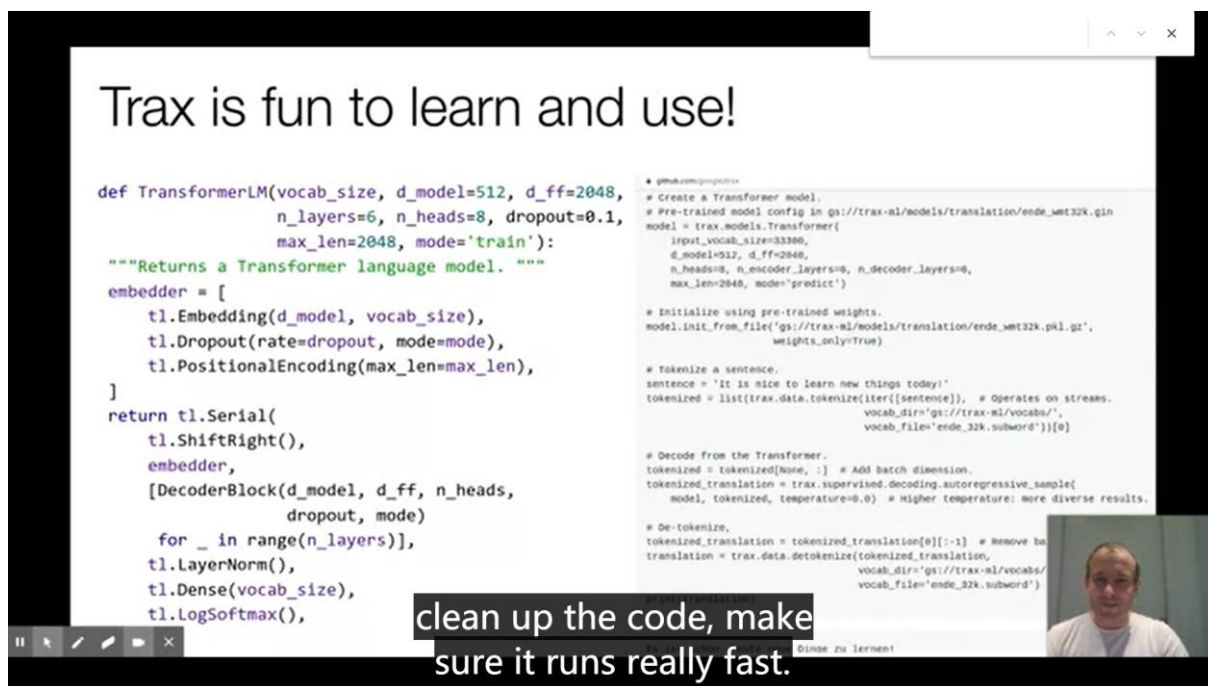
嘿，有很多漂亮的东西，你可以用几行代码编写程序，但是运行速度太慢了，实际上没有用。在 Trax 中不是这样，因为我们使用的是在 TensorFlow 过去六年中构建的即时编译器技术。它叫 XLA，我们在 Trax 上面用它。这些团队付出了巨大的努力使这件外套成为地球上最快的代码。有一种叫做 MLPerf 的行业竞争。在 2020 年，JAX 实际上赢得了这场竞争，成为有史以来独立进行基准测试的速度最快的变压器。所以 JAX-transformer 在 0.26 分钟内运行，所以我认为大约 16 秒，而同一硬件上最快的 TensorFlow 转换器需要 0.35 分钟。所以你看，它几乎慢了 50%。最快的 Pytorch，但这不在 TPU 上，取了 0.62。所以速度快两倍是一个重要的优点。目前还不清楚其他硬件上的任何型号都能获得同样的增益。有很多工作来调整它，以适应这种特殊型号的硬件。但总的来说，Trax 跑得很快。这意味着，在云计算上运行的 tpu 和 gpu 的费用会降低。它也在 Colab 上用 TPUs 测试过。colab 是谷歌免费提供给你的 IPython 笔记本。你可以选择一个硬件加速器，你可以选择 TPU，运行相同的代码而不做任何改变。它是 GPU，TPU，或 CPU，在这个 Colab 上，你可以免费得到一个 8 TPU。所以你可以在那里测试你的代码，然后在云端以比其他框架便宜得多

的价格运行它，而且它运行得非常快。所以这些是使用 Trax 的原因，对我来说，Trax 也是超级有趣的。这是超级有趣的学习，这是超级有趣的使用，因为我们有自由从零开始使用多年的经验。

The screenshot shows a presentation slide titled "Trax runs code fast". It lists several bullet points: "Designed to use a JIT compiler with JAX and XLA", "JAX: fastest Transformer in MLPerf 2020 (JAX: 0.26, TF: 0.35, pyTorch: 0.62)", "No code changes at all between CPU, GPU and TPU, preemptible training", and "Tested with TPUs on colab too:". Below the bullet points is a "Notebook settings" section with a "Hardware accelerator" dropdown menu set to "TPU". At the bottom, there is a table comparing GPU and TPU hardware costs. A video inset in the bottom right corner shows a man speaking, with the text "and it really runs fast." overlaid on the video.

Hardware	on-demand	preemptible
GPU (8x V100)	\$19.84 / hour	\$5.92 / hour
TPU (8x v3)	\$8 / hour	\$1.40 / hour

现在。你可以用组合词来写模型。左边是一个完整的 transformer 语言模型。在右边，你可以从自述中看到。这是运行预先训练的模型并获得翻译所需的一切。所以这给了我们清理框架，清理代码，确保它运行得非常快的机会。使用起来很有趣。所以我鼓励你来看看。看看你如何使用 Trax 进行自己的机器学习，两者都是为了研究。如果你想创办一家初创公司，或者你想为一家大公司经营，我认为 Trax 会一直支持你。



不得不说一句，Trax 真的很简洁，又高效！

2.4 Trax 官方文档和源码（包含 JAX 库）

由 Google Brain 团队维护的 Trax 官方文档：

<https://trax-ml.readthedocs.io/en/latest/>

GitHub 上的 Trax 源代码：

<https://github.com/google/trax>

JAX 库：

<https://jax.readthedocs.io/en/latest/index.html>

2.5 python 中的类介绍

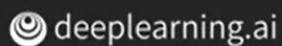
类的简单使用：

Classes in Python

```
class MyClass(Object):  
    def __init__(self, y):  
        self.y = y  
    def my_method(self, x):  
        return x + self.y  
    def __call__(self, x):  
        return self.my_method(x)
```

```
f = MyClass(7)  
print(f(3))  
10
```

Pause and check this process if you like.



子类的继承使用：

Subclasses

```
class MyClass(Object):  
    def __init__(self, y):  
        self.y = y  
    def my_method(self, x):  
        return x + self.y  
    def __call__(self, x):  
        return self.my_method(x)
```

```
class SubClass(MyClass):  
    def my_method(self, x):  
        return x + self.y**2
```

```
f = SubClass(7)  
print(f(3))  
52
```



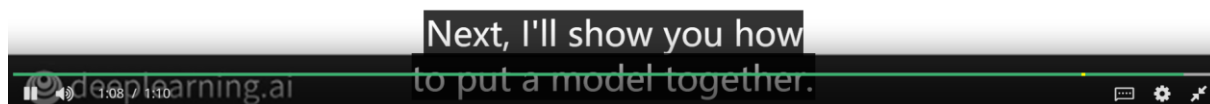
2.6 Trax 中的图层

- 稠密层
- RELU 层

分享

Summary

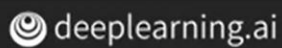
- Dense Layer $\longrightarrow z^{[i]} = W^{[i]}a^{[i-1]}$
- ReLU Layer $\longrightarrow g(z^{[i]}) = \max(0, z^{[i]})$



- 串行层（子层按照顺序排列构成串行层）

Summary

- Serial layer is a composition of sublayers



some additional layers and
the training procedure.

- 其他层

嵌入层 (将单词映射为词嵌入向量)

Embedding Layer

Vocabulary	Index		
I	1	0.020	0.006
am	2	-0.003	0.010
happy	3	0.009	0.010
because	4	-0.011	-0.018
learning	5	-0.040	-0.047
NLP	6	-0.009	0.050
sad	7	-0.044	0.001
not	8	0.011	-0.022

Trainable
weights

Vocabulary
x
Embedding



improves the weights
matrices on each layer.

均值层 (大小和嵌入维数一致)

Mean Layer

Tweet: I am happy

Vocabulary	Index		
I	1	0.020	0.006
am	2	-0.003	0.010
happy	3	0.009	0.010

0.020	0.006
-0.003	0.010
0.009	0.010

Mean of the
word
embeddings

0.009
0.009

No trainable
parameters

only computing the mean
of the word embeddings.

Summary

- Embedding is trainable using an embedding layer
- Mean layer gives a vector representation

and that's the mean layer
takes a matrix of embeddings,

2.7 Trax 框架下的梯度下降

Summary

- `grad()` allows much easier training
- Forward and backpropagation in one line!

 deeplearning.ai **train a neural network using Trax.**

梯度计算只需要一行：

Computing gradients in Trax

$$f(x) = 3x^2 + x$$

$$\frac{\delta f(x)}{\delta x} = 6x + 1$$

Gradient

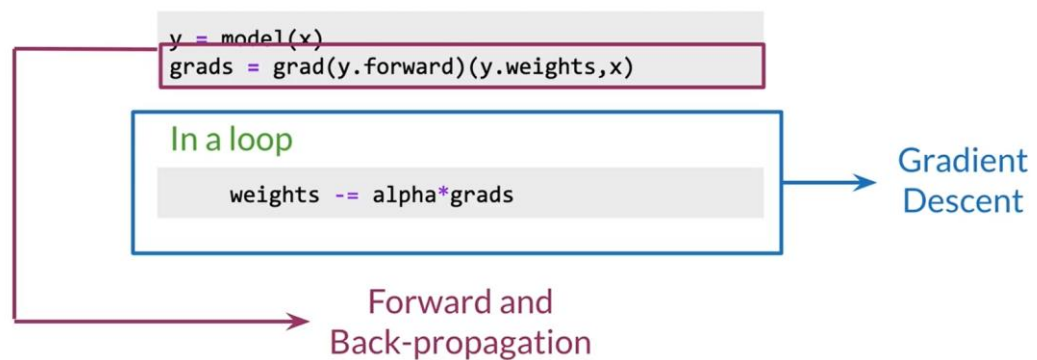
```
def f(x):  
    return 3*x**2 + x  
grad_f = trax.math.grad(f)
```

Returns a
function

 deeplearning.ai **The function grad will return a function**

模型中后向传播中梯度的计算：

Training with grad()



假设你有一个神经网络模型 y 。要得到你的模型的梯度，只需应用梯度函数和你的模型的正向方法作为一个参数。然后用权重和输入来评估渐变。注意双括号。第一个传递 `grad` 函数的参数，第二个传递 `grads` 返回的函数的参数。有了模型的渐变之后，只需迭代直到达到收敛。就是这样，前向和后向传播在一条直线上执行。

3. 第二周 (N-gram 与序列模型)

3.1 传统语言模型

上门课程讲述了经典的 N-gram 模型，用来计算生成单词序列的概率，但这种方法需要大量的内存和空间。

这些模型有很多限制。其中之一是为了捕获单词之间的依存关系 彼此相距遥远，您的模型将不得不考虑 非常长的单词序列中的条件概率。如果没有相应的大型语料库，这可能很难估计。即使是大型语料库，您的模型也需要大量空间，RAM 存储所有可能组合的概率。您会看到这种方法变得不切实际的速度。

N-grams

$$P(w_2|w_1) = \frac{\text{count}(w_1, w_2)}{\text{count}(w_1)} \longrightarrow \text{Bigrams}$$

$$P(w_3|w_1, w_2) = \frac{\text{count}(w_1, w_2, w_3)}{\text{count}(w_1, w_2)} \longrightarrow \text{Trigrams}$$

$$P(w_1, w_2, w_3) = P(w_1) \times P(w_2|w_1) \times P(w_3|w_2)$$

- Large N-grams to capture dependencies between distant words
- Need a lot of space and RAM

Up next, I'll introduce you to recurrent

deep learning neural networks and gated recurrent units.

What would be the probability of a five word sequence using a penta-gram?

- ☒ $P(w_5, w_4, w_3, w_2, w_1) = P(w_1) \times P(w_2|w_1) \times P(w_3|w_1, w_2) \times P(w_4|w_1, w_2, w_3) \times P(w_5|w_1, w_2, w_3, w_4)$
- ☐ $P(w_5|w_4, w_3, w_2, w_1) = \frac{\text{count}(w_5, w_4, w_3, w_2, w_1)}{\text{count}(w_4, w_3, w_2, w_1)}$
- ☐ $P(w_5, w_4, w_3, w_2, w_1) = P(w_1) \times P(w_2) \times P(w_3) \times P(w_4) \times P(w_5)$
- ☐ $P(w_5, w_4, w_3, w_2, w_1) = P(w_5|w_4, w_3, w_2, w_1)$

2.2 循环神经网络 (RNN) Recurrent neural network

相比于 N-gram, RNN 可以捕获更长的依赖关系。

下图中, 经典的 N-gram 很可能会选择 have, 因为在 3-gram 的模型中, did not have 出现的频率很高, 所以

Advantages of RNNs

Nour was supposed to study with me. I called her but she **did not** have

want
respond
choose
want
have
ask
attempt
answer
know

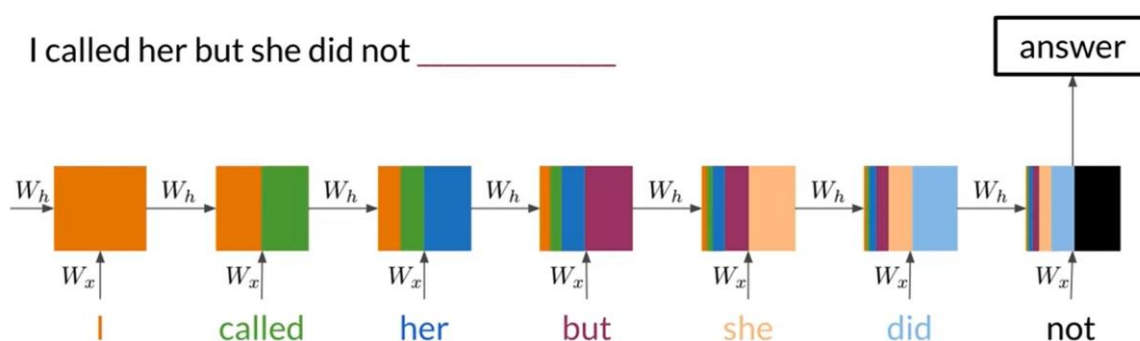
Similar probabilities with trigram

deeplearning.ai

For instance, your model
might choose the word have,

RNNs Basic Structure

I called her but she did not _____



deeplearning.ai

It is multiplied by W
subscript H. In other words,

RNN 的主要优点是它们可以传播序列中的信息、计算时共享大多数参数。在这

里，您看到了一个示例传播信息的 RNN 从头到尾传播一个单词序列，最后进行单个预测。

3.3 RNN 架构类型

- 一对一（one to one）

球星在排行榜上的位置

- 一对多（one to many）

为图片增添简介（命名实体识别）

- 多对一（many to one）

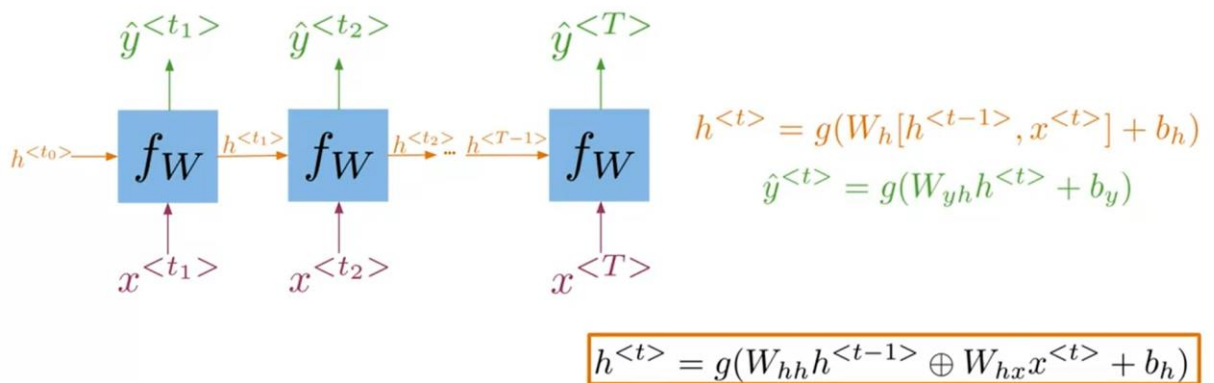
情感分析，对电影评论语句进行星级评定（五星到一星）。

- 多对多（many to many）

语言翻译、编解码器、字幕生成

3.4 简单 RNN 的教学

A Vanilla RNN

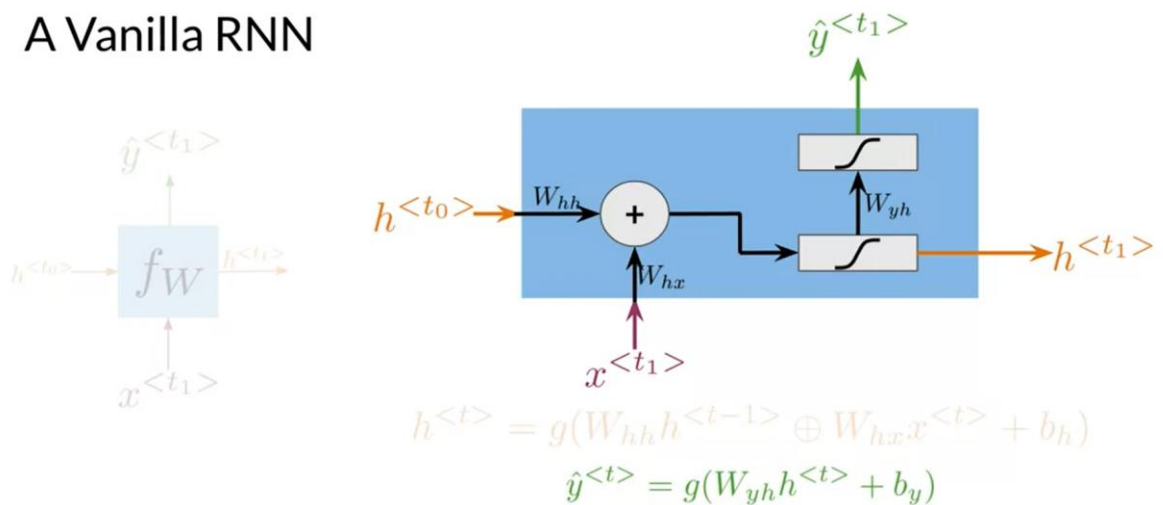


the order in which
computations are made.

deeplearning.ai

公式的计算很重要，涉及维数

A Vanilla RNN



information is propagated
with the recurrent unit.

deeplearning.ai

这道题目选什么？

试题

What should be the size of matrix W_h , if $h^{<t>}$ had size 4×1 and $x^{<t>}$ 10×1 ?

$$h^{<t>} = g(W_h[h^{<t-1>}, x^{<t>}] + b_h)$$

- ☐ 14×4
- ☒ 4×14
- ☐ 14×14
- ☐ 4×4

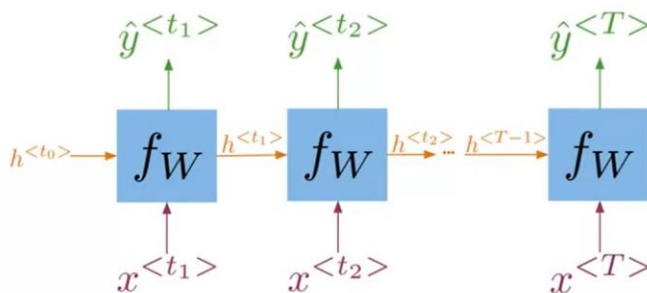
✓ 正确

跳过

继续

RNN 的代价函数

Cross Entropy Loss



$$h^{<t>} = g(W_h[h^{<t-1>}, x^{<t>}] + b_h)$$
$$\hat{y}^{<t>} = g(W_{y_h}h^{<t>} + b_y)$$

$$J = -\frac{1}{T} \sum_{t=1}^T \sum_{j=1}^K y_j^{<t>} \log \hat{y}_j^{<t>}$$

the summation over time t and the division

by the total number of steps, capital T .

为什么只对时间 T 求均值，而不对类别数求均值呢？ 这是个好问题：

试题

In the next equation, why is there a division by the number of time steps but not one for the number of classification categories?

$$J = -\frac{1}{T} \sum_{t=1}^T \sum_{j=1}^K y_j^{<t>} \log \hat{y}_j^{<t>}$$

- ☒ Because there is just one value in every vector $y^{<t>}$ different from zero
- ☐ Because the equation is wrong
- ☐ Because this equation is given for a single example
- ☐ Because for most classification tasks there are only two categories

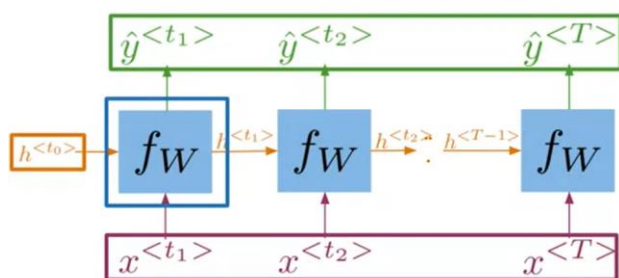
✓ 正确

因为我们的类别取值只有 0 或 1，这两个标签。

RNN 的流程图：

在 tensorflow 中实现功能扫描，用于计算 RNN 中的前向传播。

tf.scan() function



```
def scan(fn, elems, initializer=None, ...):  
    cur_value = initializer  
    ys = []  
    for x in elems:  
        y, cur_value = fn(x, cur_value)  
        ys.append(y)  
    return ys, cur_value
```



deeplearning.ai

Finally, the function returns the list of predictions, and the last hidden state.

题目测试：问题是对的

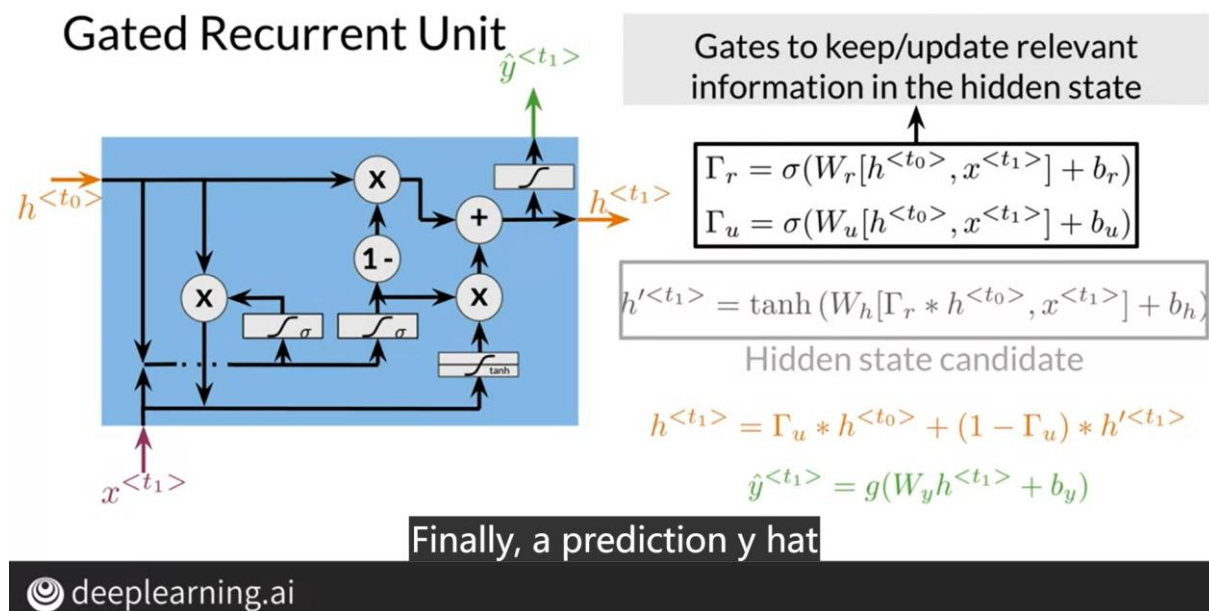
试题

In the scan() function the variable cur_value corresponds to the hidden state in an RNN.

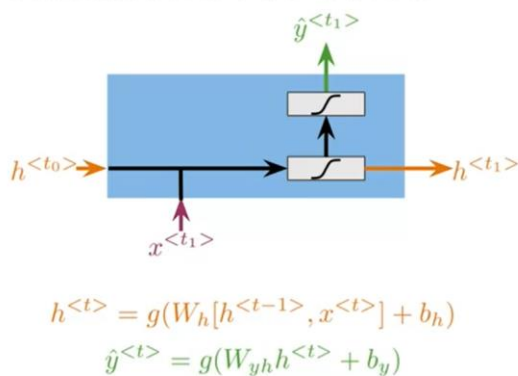
```
def scan(fn, elems, initializer=None, ...):  
  
    cur_value = initializer  
    ys = []  
  
    for x in elems:  
        y, cur_value = fn(x, cur_value)  
        ys.append(y)  
  
    return ys, cur_value
```

3.5 GRU (门控循环单元)

与普通的 RNN 相比，gru 的工作方式允许相关信息保持在隐藏状态，即使是长序列。也可以将 GRU 看作是附带额外计算的 RNN。

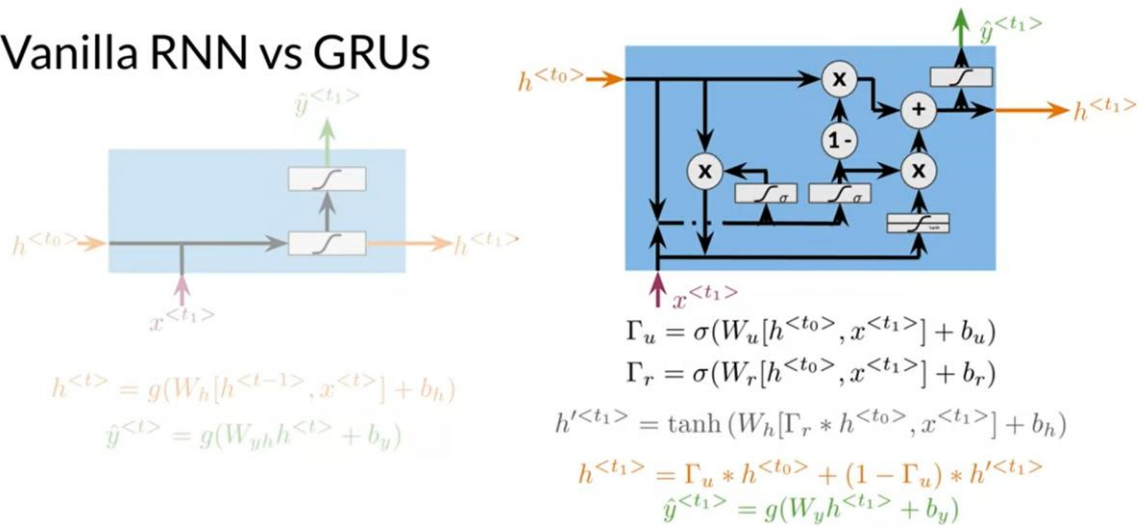


Vanilla RNN vs GRUs



This is one cause of

Vanilla RNN vs GRUs



deeplearning.ai

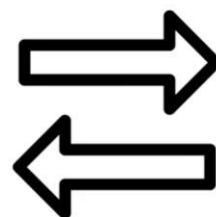
All of these computations
allow the network to learn

深度递归神经网络非常有用，因为它们可以让您捕获 使用浅层 RNN 无法捕获的依赖项。

3.6 深层 RNN 和双向 RNN

Outline

- How bidirectional RNNs propagate information
- Forward propagation in deep RNNs

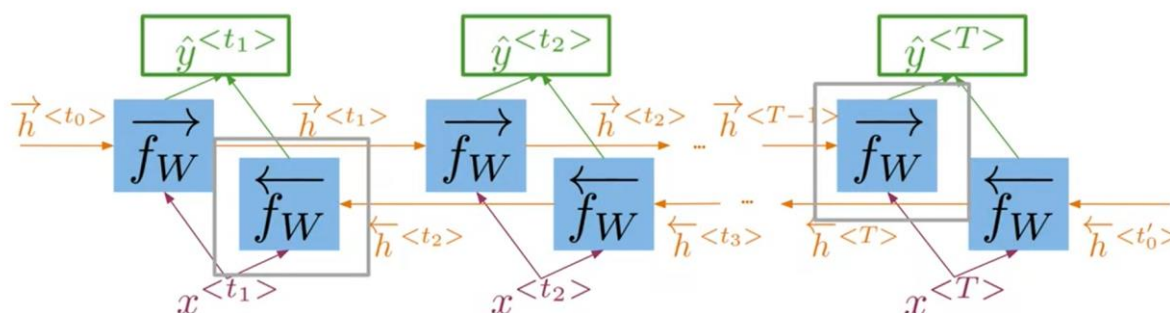


I will show you how bidirectional
neural networks work and



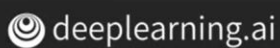
双向 RNN，从左到右和从右到左的计算是独立的，这不是个循环图。

Bi-directional RNNs



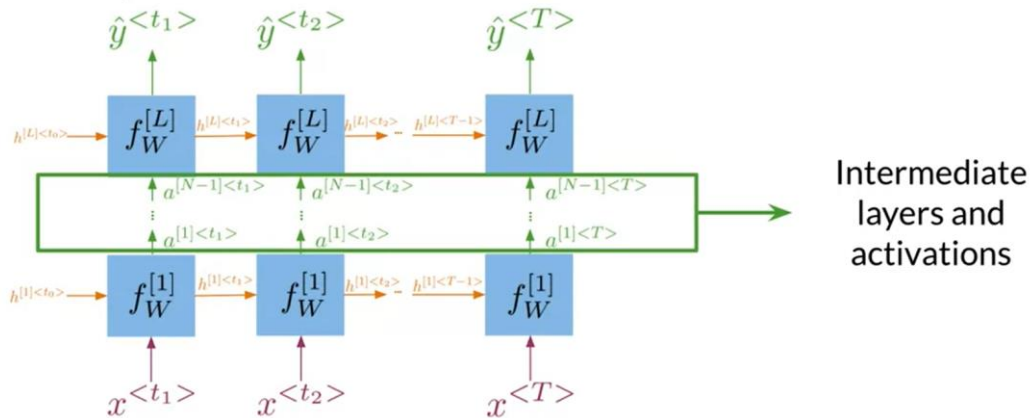
$$\hat{y}^{<t>} = g(W_y[\vec{h}^{<t>}, \overleftarrow{h}^{<t>}] + b_y)$$

both directions, you can get all
of the remaining predictions.



深度 RNN:

Deep RNNs



As you can see,

deep RNNs are just RNNs stuck together.

总结:

Summary

- In bidirectional RNNs, the outputs take information from the past and the future
- Deep RNNs have more than one layer, which helps in complex tasks



Deep RNNs can help you solve more complex

tasks that aren't possible with shallow

4. 第三周

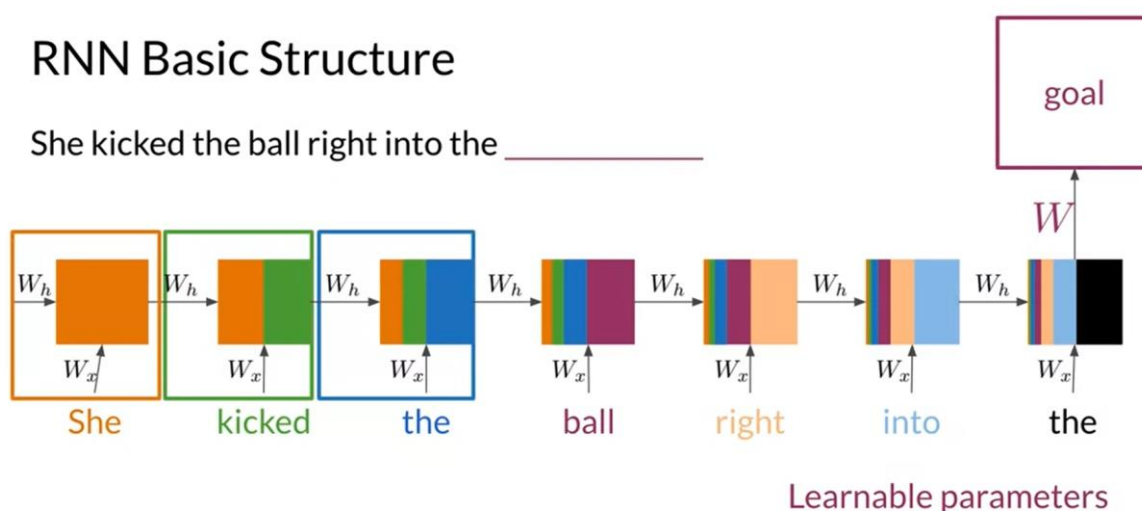
LSTM 和 命名实体识别

4.1 RNN 和梯度消失

- 普通的 RNN 优缺点
 - 一定程度上能捕获前面文本的信息，与 N-gram 相比，更加轻量级，占用更少的 RAM 和空间。
 - 但是保留的文本信息会随着距离变长而逐渐变少，因此它难以处理较长文本序列。随着时间的增长或层数的增多，容易产生梯度消失或者梯度爆炸。导致训练失败。

RNN Basic Structure

She kicked the ball right into the _____



deeplearning.ai

have much influence on
the cost function either.

所以，第一步（第一个单词的处理）进行的计算不会对代价函数有较大的影响。梯度是在反向传播过程中计算的，或者是从前向过程中到达的最终层向后移动到初始层的过程。每层的导数从后向前相乘，以计算初始层的导数。可以将渐变视为模型在一系列时间步中可以改进多少的度量。当您的网络执行反向传播时，它的权重接收一个更新，它与相对于该时间步的当前权重的梯度成比例。但是在具有多个时间步长或多个层的网络中，由于后一层所有项的乘积，使得一个梯度回到早期层，从而导致了固有的不稳定情况。尤其是如果值变得如此小，这就是它不再正确更新的原因。对于渐变爆炸的问题，想象一下这个过程在相反的方向上工作，因为

更新后的权重变得如此之大，以至于导致整个网络变得不稳定。这种情况会导致数值溢出。

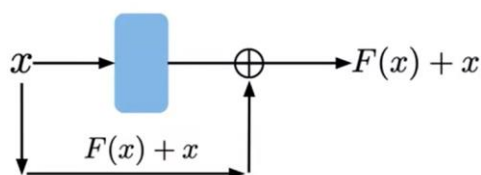
- 梯度问题的解决办法：

Solving for vanishing or exploding gradients

- Identity RNN with ReLU activation $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$ $-1 \rightarrow 0$

- Gradient clipping $32 \rightarrow 25$

- Skip connections



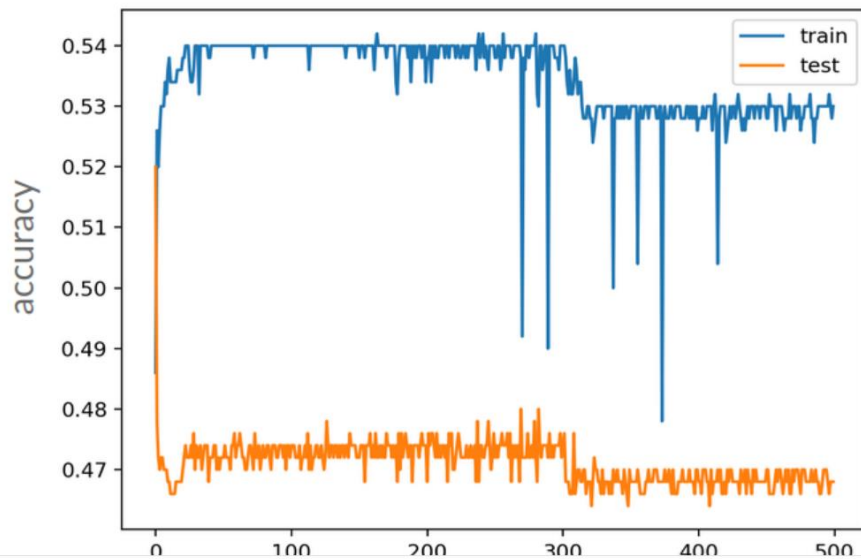
activations from early layers have

deeplearning.ai more influence over the cost function.

- 你可以通过初始化你的权重到单位矩阵来处理渐变的消失，这个矩阵沿着主对角线携带 1 的值，在其他任何地方都是 0，并使用 ReLU 激活。它的实质是复制以前的隐藏状态，从当前输入中添加信息，并用 0 替换任何负值。这有助于鼓励你的网络保持接近单位矩阵中的值，在矩阵乘法过程中这些值的作用类似于 1。这种方法被称为身份 RNN，这并不奇怪。仅适用于通过消除渐变。若要考虑指数增长的值，可以执行渐变剪裁。
- 要剪裁渐变，只需选择一个将渐变剪裁到的相关值，例如 25。使用此技术，任何大于 25 的值都将被剪裁为 25，这有助于限制渐变的大小。
- 最后，跳过连接提供到早期层的直接连接。这有效地跳过了激活函数，并将初始输入 x 的值添加到输出值，即 $F(x) + x$ 。这样，早期层的激活对成本函数的影响更大。

试题

This plot shows poor model performance and may indicate a vanishing gradient problem.



正确!

如果您有兴趣更深入地了解梯度下降中的一些挑战，请查看 Paperspace.io 上的此博客。

<https://blog.paperspace.com/intro-to-optimization-in-deep-learning-gradient-descent/>

4.2 LSTM

LSTMs: a memorable solution

- Learns when to remember and when to forget
- Basic anatomy:
 - A cell state
 - A hidden state with three gates
 - Loops back again at the end of each time step
- Gates allow gradients to flow unchanged

The risk of vanishing or



LSTM 本质上是由单元状态组成的，您可以将其视为记忆，和隐藏状态。在其中执行计算的地方，进行训练以决定要进行哪些更改。在再次执行整个操作之前，隐藏状态通常要经过三个门，任何循环都会更新权重。这些单元状态通过这三个门可以跟踪输入的到达。每个人都参与决策，传递多少信息以及保留多少。这一系列的门控单元允许梯度不变地流动。梯度消失或爆炸的风险逐渐得到缓解。

以打电话的例子说明 LSTM 的三个门控单元：

LSTMs: Based on previous understanding

Cell state = before conversation

Forget gate = beginning of conversation

Input gate = thinking of a response

Output gate = responding

Updated cell state = after conversation

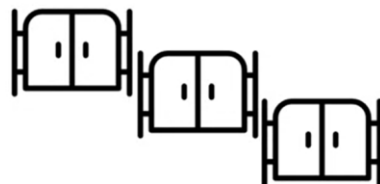


since you began
your conversation.



Summary

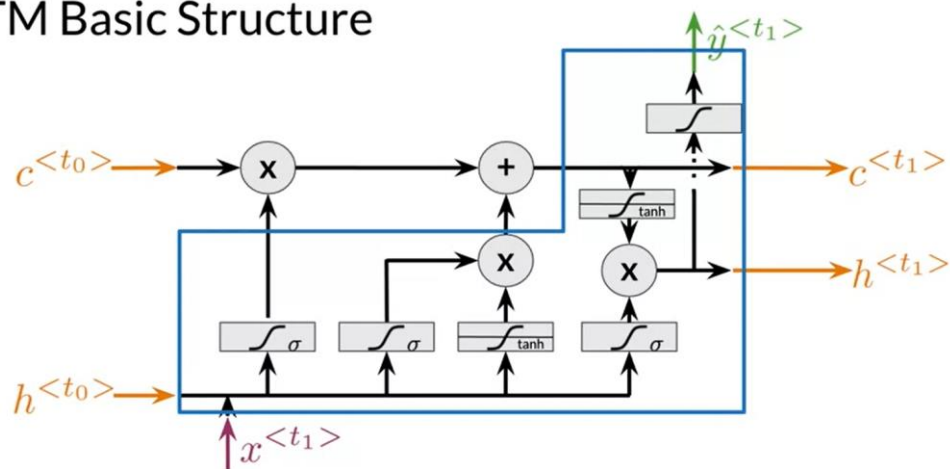
- LSTMs offer a solution to vanishing gradients
- Typical LSTMs have a cell and three gates:
 - Forget gate
 - Input gate
 - Output gate



deeplearning.ai

That typical LSTMs have a cell or memory and three gates,

LSTM Basic Structure



deeplearning.ai

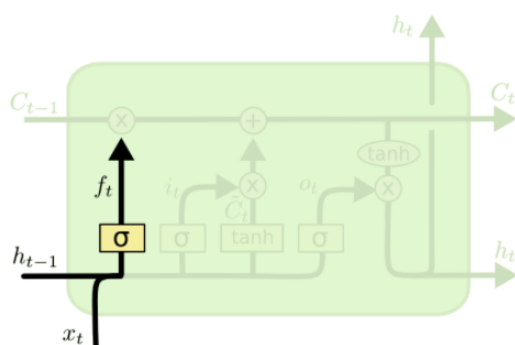
much to write to the next hidden state.

LSTM 简介的文章: <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

4.3 LSTM 体系结构

- 遗忘门（保留旧信息）

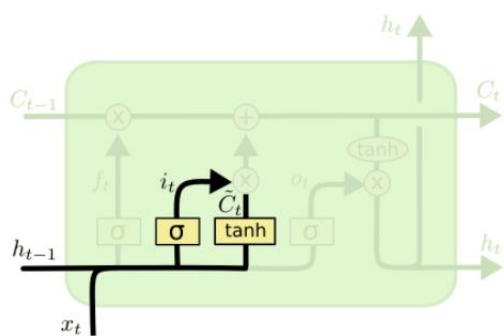
决定上个单元信息和当前输入信息的丢弃和保留。使用 sigmoid 函数近似 0 或 1 进行丢弃和保留。



$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

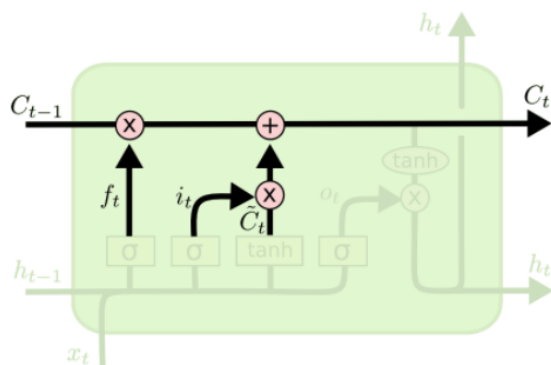
- 输入门（保留信息结合输入的当前信息进行信息更新）

实际上有两层，一层是 sigmoid 层，一层是 tanh 层。sigmoid 层决定了先前单元和当前信息有哪些需要更新，通过分配 0 或 1 决定去留。tanh 层将先前单元和当前信息压缩到 -1 到 1 之间，有助于调节网络的信息流。然后将 sigmoid 层和 tanh 层相乘，得到新的单元状态 (C_t)。



$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$



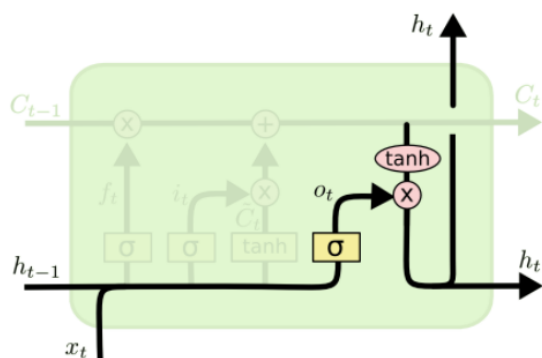
$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

- 输出门 (预测)

第一步，利用之前的隐藏状态信息生成选择，即更新了遗忘门的信息。(sigmoid 层)

第二步，利用 tanh 层将更新信息压缩。

第三步，将 sigmoid 层和 tanh 层相乘进行选择，决定了输出内容 h_t 。



$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

Summary

- LSTMs use a series of gates to decide which information to keep:
 - Forget gate decides what to keep
 - Input gate decides what to add
 - Output gate decides what the next hidden state will be



deeplearning.ai

The input gate
decides what to add.

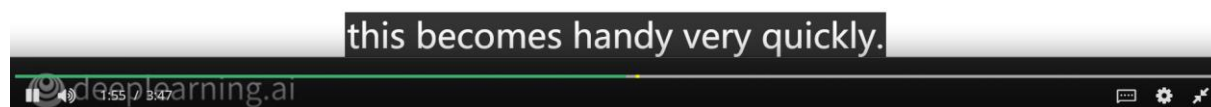
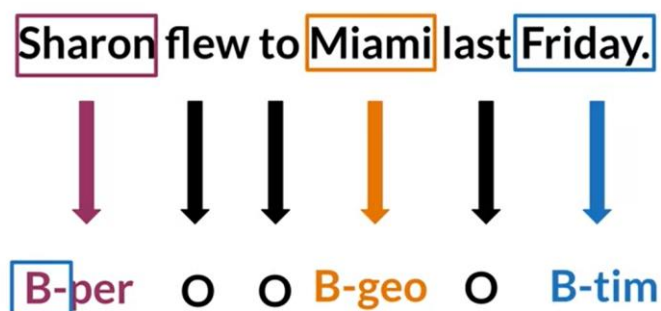
4.4 命名实体识别

命名实体识别有什么用？

优化搜索引擎效率，推荐引擎、顾客服务、自动交易。

分享

Example of a labeled sentence

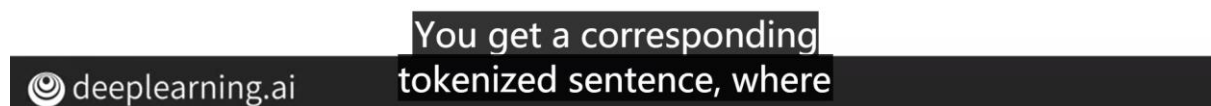
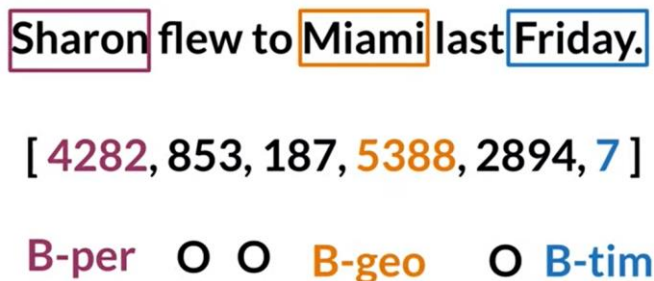


NER 的训练：数据预处理很重要

先将输入转化为定长的向量：

Processing data for NERs

- Assign each class a number
- Assign each word a number



Training the NER

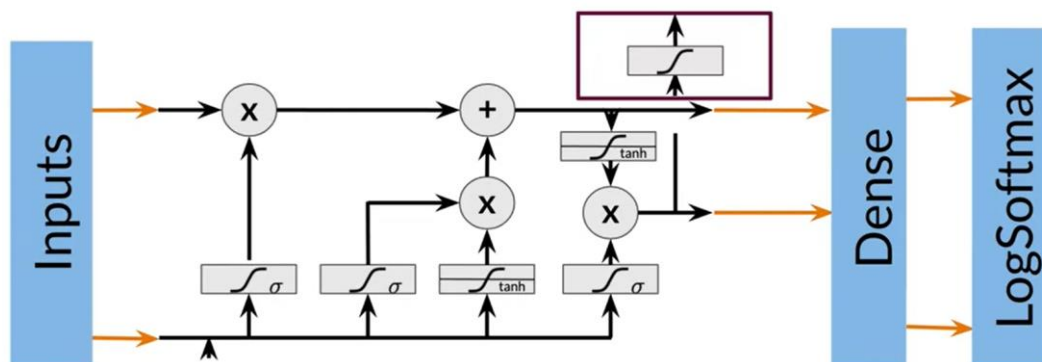
1. Create a tensor for each input and its corresponding number
2. Put them in a batch —————→ 64, 128, 256, 512 ...
3. Feed it into an LSTM unit
4. Run the output through a dense layer
5. Predict using a log softmax over K classes

gets better in numerical performance and
gradients optimization.

deeplearning.ai

分享

Training the NER



And then you compute a log softmax
to get the corresponding outputs.

deeplearning.ai

分享

在 Trax 中的使用：

Layers in Trax

```
model = tl.Serial(  
    tl.Embedding(),  
    tl.LSTM(),  
    tl.Dense(),  
    tl.LogSoftmax()  
)
```



试题

Why do all sequences need to be the same length for processing through an LSTM unit?

- ☒ In a vectorized representation of your data, equal sequence length allows more efficient batch processing.
- ☐ This is what allows the gradients to flow unchanged through the LSTM unit.

✓ 正确

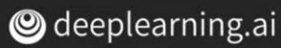
跳过

继续

评估的时候，请记得你做的是个批处理生成器，所以 next () 很重要。

Evaluating the model in Python

```
def evaluate_model(test_sentences, test_labels, model):  
    pred = model(test_sentences)  
    outputs = np.argmax(pred, axis=2)  
    mask = ...  
    accuracy = np.sum(outputs==test_labels)/float(np.sum(mask))  
  
    return accuracy
```



Which built-in Python method would you use to iterate over your test set during the evaluation step?

- ☐ list()
- ☐ slice()
- ☒ next()
- ☐ enumerate()

✓ 正确

5. 第四周

Siamese 网络

5.1 Siamese 网络简介

比较含义并不像比较单词那么简单。在这些平台允许您发布新问题之前，他们希望确保您的问题尚未由其他人发布。这正是 Siamese 存在的意义。

Question Duplicates

How old are you? = What is your age?

Where are you from? $\neq$ Where are you going?

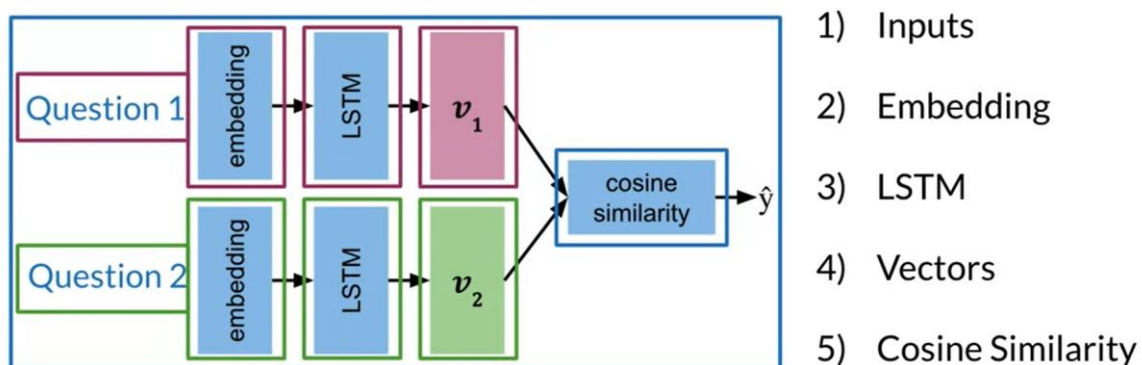
at the core of platforms like
Stack Overflow or Quora.

借助于 Siamese 网络，您将着眼于确定什么使两个输入相似，是什么让他们与众不同。应用场景也很多：搜索引擎对问题的筛选分类，手写签名检查、识别 Quora 或 堆栈溢出等。

5.2 架构

Siamese 网络具有特殊类型的架构。他们有两个相同的子网，合并在一起，通过磁盘层产生最终输出或其相似性得分。子网络共享参数，因此只需要训练一组网络就可以对两个输入向量进行相似度判断。

Model Architecture



compare them using cosine
similarity to get your $\hat{y}$.

5.3 代价函数

Loss Function

按 Esc 即可退出全屏模式

How old are you?

Anchor

$$\cos(v_1, v_2) = \frac{v_1 \cdot v_2}{||v_1|| ||v_2||}$$
$$s(v_1, v_2)$$

What is your age?

Positive

$$s(A, P) \approx 1$$

Where are you from?

Negative

$$s(A, N) \approx -1$$

$$s(A, N) - s(A, P)$$

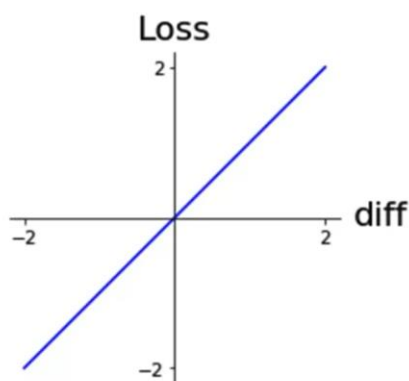
you start with the

deeplearning.ai

similarity of A and N and

分享

Loss Function



$$\text{diff} = s(A, N) - s(A, P)$$

is roughly doing what

deeplearning.ai

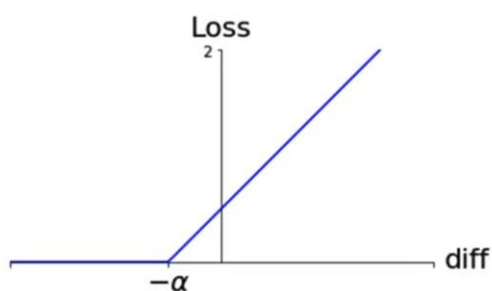
you hope it will do.

2:40 / 3:13

5.4 三胞胎?

即将目标文本，正例文本和负例文本组成一个三元组，也称三胞胎。三重损失，即计算这个三元组的损失函数。

Triplet Loss



Simple loss:

$$\text{diff} = s(A, N) - s(A, P)$$

Non linearity:

$$\mathcal{L} = \begin{cases} 0; & \text{if } \text{diff} \leq 0 \\ \text{diff}; & \text{if } \text{diff} > 0 \end{cases}$$

Alpha margin:

$$\mathcal{L} = \begin{cases} 0; & \text{if } \text{diff} + \alpha \leq 0 \\ \text{diff} + \alpha; & \text{if } \text{diff} + \alpha > 0 \end{cases}$$

to the left by the amount Alpha.

 deeplearning.ai

假设我们选择 Alpha 为 0.2，如果相似度之间的差异很小，如负 0.1，然后如果将其添加到 Alpha 0.2，结果仍然大于 0。差异加 Alpha 的总和可以考虑正面损失说明此示例中学习的模型。您可以在折线图中直观地看到这一点。损失函数转移向左按 Alpha 数量。差异沿水平轴。当差异小于零但幅度很小，损失大于零。因此，如果差异的大小小于 Alpha，那么仍然有损失。这种损失告诉模型它仍然可以改进并从该培训示例中学习。

试题

In the triplet cost function below, will increasing the hyperparameter alpha from 0.2 to 0.5 require more, or less, optimization during training ?

$$\max (-\cos(A, P) + \cos(A, N) + \alpha, 0)$$

☐ Less

☒ More

✓ 正确

Triplet Selection

Hard triplets are better for training !

Triplet A, P, N:  duplicate set: A, P
non-duplicate set: A, N



Random: $\mathcal{L} = \max (diff + \alpha, 0)$
 $diff = s(A, N) - s(A, P)$
Easy to satisfy. Little to learn



Hard: $s(A, N) \approx s(A, P)$
Harder to train. More to learn
the similarity between
anchor and positive.



 deeplearning.ai

不要选择随机的 (A, N) 例子来进行三元组的训练。如上图所示, 随机的负例很容易满足损失函数的优化, 但是很难去学习改进参数。您需要专门选择所谓的三胞胎, 也就是说, 更难以训练的三胞胎。硬三胞胎是那些相似之处 锚定 (anchor) 和负数之间非常接近, 但仍然小于锚与正之间的相似性。当模型遇到硬三元组时, 学习算法需要调整其权重, 这样就可以产生相似性 与现实世界中的标签保持一致。

5.5 代价函数的计算

设置了两个批次，每个批次大小是 4（行）， V 列（嵌入维度）

Computing The Cost

Prepare the batches as follows:

What is your age?

Can you see me?

Where are thou?

When is the game?

How old are you?

Are you seeing me?

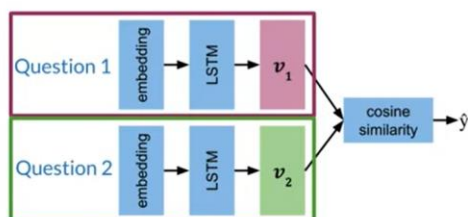
Where are you?

What time is the game?

$b = 4$

want to organize the
data in this way.

Computing The Cost

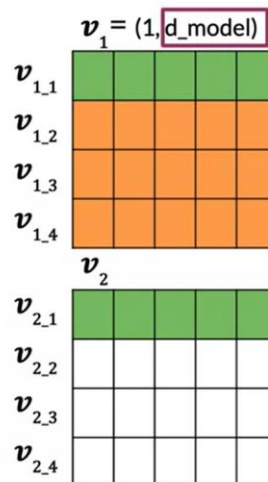


Batch 1

What is your age?
Can you see me?
Where are thou?
When is the game?

Batch 2

How old are you?
Are you seeing me?
Where are you?
What time is the game?



all vector pair combinations
of v_1 with v_2 .

Computing The Cost

	$s(v_1, v_2)$			
	v_1			
	_1	_2	_3	_4
v_2				
_1	0.9	-0.8	0.3	-0.5
_2	-0.8	0.5	0.1	-0.2
_3	0.3	0.1	0.7	-0.8
_4	-0.5	-0.2	-0.8	1.0

$$\mathcal{L}(A, P, N) = \max(\text{diff} + \alpha, 0)$$
$$\text{diff} = s(A, N) - s(A, P)$$

$$\mathcal{J} = \sum_{i=1}^m \mathcal{L}(A^{(i)}, P^{(i)}, N^{(i)})$$



vastly improve upon
model performance.

注意，对于负例，其相似度仍有大于 0 的情况，这是正常的。并没有一个标准，界定了正例就必须大于 0，负例就必须小于 0，只需要正例的相似度高于负例的相似度就行。上图的这种负例设置，就不再需要额外的输入数据构建负例对。这是很重要的一个举措！

接下来注意，批次大小指的是矩阵的行数，也就是文本句子的个数。矩阵的列指的是向量的嵌入维数，引入两个新的概念，

那就是负例均值：

指每一行非对角线的均值，可以帮助损失函数加快训练速度。

最近负值：

由于需要硬三元组进行训练，所以需要寻找和正例相似度不大的负例。在每一行中寻找和正例相似度差异不大的例子，即寻找每一行非对角线上的元素值，该值和对角线上的元素值差异不大。

Hard Negative Mining

$s(v_1, v_2)$

	v_1				
	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>	
v_2	<u>1</u>	0.9	-0.8	0.3	-0.5
	<u>2</u>	-0.8	0.5	0.1	-0.2
	<u>3</u>	0.3	0.1	0.7	-0.8
	<u>4</u>	-0.5	-0.2	-0.8	1.0

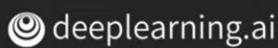
mean negative:

mean of off-diagonal values in each row

closest negative:

off-diagonal value closest to (but less than) the value on diagonal in each row

What this means is that this negative



example with a similarity of 0.3 has

损失函数的几种定义：

通过仅对几个观测值的平均值进行训练，可以减少噪声，而不是针对这些非对角线示例训练模型。那么，为什么要取几个观测值的平均值通常会减少噪声呢？好吧，我们将噪声定义为很小的值来自以 0 为中心的分布。因此，换句话说，几个噪声值的平均值通常为 0。因此，如果我们取几个示例的平均值，这具有消除那些观测中的单个噪声的效果。

Hard Negative Mining

mean negative: mean of off-diagonal values

closest negative: closest off-diagonal value

$$\mathcal{L}_{\text{Original}} = \max(\underbrace{s(A, N) - s(A, P)}_{\text{diff}} + \alpha, 0)$$

$$\mathcal{L}_1 = \max(\text{mean_neg} - s(A, P) + \alpha, 0)$$

$$\mathcal{L}_2 = \max(\text{closest_neg} - s(A, P) + \alpha, 0)$$

If you had that small difference to alpha,



then you're able to

您可以将最接近的负例，视为使得两个余弦相似度之间的最小差异的硬负例。如果您与 α 的差异很小，那么您就可以在该行的所有其他示例中产生的损失最

大。通过将培训重点放在产生更高损失值的示例上，您可以使模型更新其权重。要从这些更困难的例子中学习，那么您可以将全部损失定义为损失 1 + 损失 2。您将使用此新的全损作为分配中改进的三重损失。

分享

按 Esc 即可退出全屏模式

Hard Negative Mining

$$\mathcal{L}_{\text{Full}}(A, P, N) = \mathcal{L}_1 + \mathcal{L}_2$$

$$\mathcal{J} = \sum_{i=1}^m \mathcal{L}_{\text{Full}}(A^{(i)}, P^{(i)}, N^{(i)})$$

individual losses over the training sets.



deeplearning.ai

6:20 / 6:36



5.6 分类与一次性学习

分类与一次性学习是有区别的，考入一个简单的 K 分类问题，我们可以使用 softmax 函数对预测概率最大的那个进行选择，最终可以在已有的备选列表中找到我们的分类具体结果。但是，如果新增了一个分类又该怎么办？需要重新训练模型吗？这样的话，除非你有很多新例子添加进来，不然模型的训练仍旧是不尽人意。

一次性学习，只需要识别一个例子就可以反复进行签名认证，可以使用相似度函数和设定的阈值进行比较分类。类似于人脸识别，只需要一开始录入一个人的人脸信息并存储在数据库中，接下来只需要对新的人脸生成向量信息并进行数据库中的人脸比对即可。

5.7 如何训练测试 Siamase 网络

本节您将看到什么样的数据集看起来像是一个 Siamase 网络，如何训练模型，如何使用该模型测试你的 Siamase 网络。

本节给的数据集长相如下：

Dataset

Question 1	Question 2	is_duplicate
What is your age?	How old are you?	true
Where are you from?	Where are you going?	false
⋮	⋮	⋮

plenty of examples to learn from.

 deeplearning.ai

第一步，将数据集处理成如下的样子：

Prepare Batches

Question 1:
batch size b

Batch 1

What is your age?
Can you see me?
Where are thou?
When is the game?

$v_1 = (1, d_{\text{model}})$

$v_{1,1}$				
$v_{1,2}$				
$v_{1,3}$				
$v_{1,4}$				

Question 2:
batch size b

Batch 2

How old are you?
Are you seeing me?
Where are you?
What time is the game?

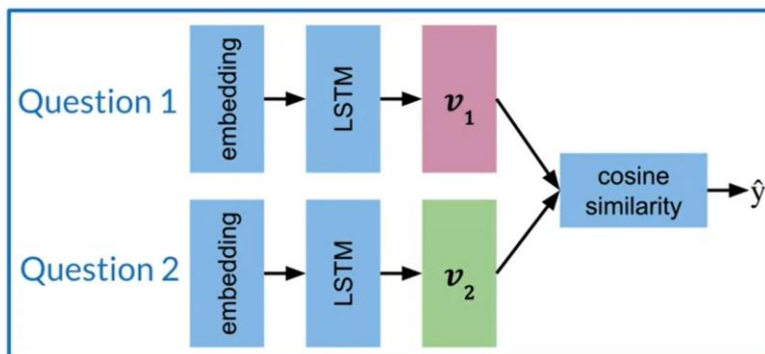
v_2

$v_{2,1}$				
$v_{2,2}$				
$v_{2,3}$				
$v_{2,4}$				

Finally, you'll use these inputs

第二步，创建 Siamese 子网进行嵌入训练：

Siamese Model



Create a subnetwork:

- 1) Embedding
- 2) LSTM
- 3) Vectors
- 4) Cosine Similarity

the same between the
two subnetworks.

第三步，测试：

Testing

1. Convert each input into an array of numbers
2. Feed arrays into your model
3. Compare v_1, v_2 using cosine similarity
4. Test against a threshold τ



deeplearning.ai

the last function are
tunable hyperparameters.